

SOMMERSEMESTER 2025

Studiengang/Abschluss: Wirtschaftsinformatik / B.Sc.
Prüfungsfach: Programmierung / Programmierung 2
Prüfungsnummer: **5706 / 55743**
Prüfer: Prof. Dr. Andreas Biesdorf
Anzahl der Arbeitsblätter: 8 Seiten (ohne Deckblätter)
Bearbeitungszeit: 90 Minuten

Mit Bleistift oder in roter Farbe geschriebene Ausführungen werden nicht gewertet!

Zugelassene Hilfsmittel:

Das im Rahmen der Vorlesung erstellte Hilfsmittel.

Matrikelnummer:

--	--	--	--	--	--

Datum der Klausur:

..... **Platznummer:**

Punkte: _____ von **60** möglichen

Note: _____

Handzeichen des Prüfers: _____

Hinweise zu Klausuren im FB Wirtschaft

- Mit Aufnahme der Prüfung bringen Sie zum Ausdruck, dass Sie sich für prüfungstauglich halten. Ein Rücktritt von der Klausur während der Bearbeitungszeit ist damit nur noch in ganz besonderen Ausnahmefällen möglich.
- Sollten Sie zu Beginn der Bearbeitungszeit prüfungstauglich sein, dann aber während der Bearbeitungszeit prüfungsuntauglich werden, müssen Sie die Klausur abbrechen, Ihre Prüfungsuntauglichkeit den Aufsichtführenden anzeigen und schnellstmöglich, in jedem Fall aber fristgerecht für ein ärztliches Attest sorgen.
- Mit der Abgabe Ihrer Klausur bringen Sie zum Ausdruck, dass Sie sich während der gesamten Bearbeitungszeit für prüfungstauglich gehalten haben. Ein Rücktritt ist dann nicht mehr möglich.
- Jacken und Taschen sind in ausreichender Entfernung vom Platz zu deponieren.
- Die Bearbeitungszeit beginnt erst, wenn alle Klausuren ausgeteilt sind.
- Es sind nur die zugelassenen Hilfsmittel zu verwenden.
- Das Verlassen des Raumes für Toilettengänge während der Klausur ist nur nach Meldung und Bestätigung der Aufsicht gestattet. Die vorübergehende Abwesenheit wird im Protokoll vermerkt.
- Es sind keine anderen als die ausgegebenen Bearbeitungsblätter zu benutzen.
- Auf dem Tisch sind nur Schreibutensilien erlaubt, keine Mäppchen.
- Jeder Teilnehmer muss sich mit einem Studierendenausweis oder Lichtbildausweis ausweisen.
- Geräusche und Störungen jeder Art sind zu vermeiden.
- Die Klausurensätze sind grundsätzlich nicht zu öffnen. Wer den Klausurensatz öffnet, trägt dafür Sorge, dass die Blätter (innerhalb der Bearbeitungszeit) sortiert und neu geheftet werden. Das Risiko des Verlustes von einzelnen Blättern trägt der Studierende!
- Es ist nicht erlaubt, Aufgabenstellungen auf gesondertes Papier abzuschreiben.
- Mobiltelefone, Smartphones und Smartwatches sind nicht erlaubt!
- Alle Studierenden bleiben auf ihren Plätzen und verhalten sich ruhig, bis die Klausuren vollständig eingesammelt sind. Folgen Sie den Anweisungen der Aufsichtführenden.
- Bei Täuschung oder versuchter Täuschung wird mit „nicht ausreichend“ bewertet.

Die Klausur besteht aus 6 Aufgaben zu den in der Lehrveranstaltung behandelten Themenbereichen. Die Punktzahl ist bei jeder Aufgabe mit angegeben.

Aufgabe	Mögliche Punkte	Erreichte Punkte
A1	6	
A2	15	
A3	10	
A4	17	
A5	12	
Summe	60	
Evtl. Bonus		

Bitte dokumentieren Sie die Antworten zu allen Aufgaben in der entsprechenden Textbox auf dem Blatt oder der entsprechend benannten Datei (z.B. aufgabe1.txt oder aufgabe5.py) in dem Ordner auf der VM.

Aufgabe 1

- a) Erläutern Sie den Unterschied zwischen „Softwaretechnik“ und einfachem „Programmieren“. Gehen Sie dabei auf mindestens zwei Aspekte ein und nennen und erklären dabei kurz wesentliche Merkmale, die Softwaretechnik vom reinen Codieren unterscheiden. (3 Punkte)

- b) Nennen und erläutern Sie die Hauptrollen in Scrum. Beschreiben Sie außerdem die zentralen Schritte im typischen Scrum-Prozess. (3 Punkte)

Aufgabe 2

1) Gegeben sei folgende Dokumentation einer Funktion:

prog_ss25/src/aufgabe02a.py

```
def finde_groesste_gerade_zahl(zahlenliste):
    """
    Bestimmt die größte gerade Zahl in einer gegebenen Liste von Ganzzahlen.

    Parameter:
    -----
    zahlenliste: list of int
        Eine Liste von Ganzzahlen. Die Liste kann beliebig viele Elemente
        enthalten, aber auch leer sein.

    Rückgabewert:
    -----
    int oder str
        - Gibt die größte gerade Zahl in der Liste zurück, falls mindestens
          eine gerade Zahl vorhanden ist.
        - Gibt den String "Keine gerade Zahl gefunden" zurück, falls keine
          gerade Zahl in der Liste vorhanden ist.

    Hinweise:
    -----
    - Die Reihenfolge der Elemente in der Liste ist beliebig.
    - Negative gerade Zahlen werden ebenfalls berücksichtigt.
    - Bei mehrfach vorkommenden gleichen geraden Zahlen wird der Wert nur
      einmal zurückgegeben.
    """
```

a) Benennen Sie fünf möglichst überschneidungsfreie Äquivalenzklassen zum Testen dieser Funktion und begründen Sie Ihre Wahl stichwortartig **(4 Punkte)**

b) Erstellen Sie in der Datei prog_ss25/test/test_aufgabe02a.py die korrespondierenden Tests mithilfe von pytest. **(3 Punkte)**

2) Gegeben sei folgendes Programm:

prog_ss25/src/aufgabe02b.py

```
def schnittmenge(menge1, menge2):
    """
    Bestimmt die Schnittmenge zweier Listen, indem alle Elemente,
    die in beiden Listen vorkommen, gesammelt werden.

    Die Funktion arbeitet rekursiv und prüft jedes Element aus `menge1`,
    ob es auch in `menge2` vorhanden ist. Die Reihenfolge der gefundenen
    Elemente entspricht der Reihenfolge in `menge1`.

    Parameter:
    -----
    menge1 : list
        Eine Liste von beliebigen Objekten. Die Liste kann leer sein.

    menge2 : list
        Eine weitere Liste von beliebigen Objekten. Die Liste kann leer sein.

    Rückgabewert:
    -----
    list
        Eine Liste aller Elemente, die sowohl in `menge1` als auch in `menge2`
        vorkommen. Falls keine gemeinsamen Elemente existieren, wird eine leere
        Liste zurückgegeben.

    Hinweise:
    -----
    - Die Funktion prüft jedes Element einzeln und rekursiv.
    - Mehrfache Vorkommen eines Elements in `menge1` werden mehrfach in der
      Rückgabe enthalten,
      sofern das Element in `menge2` existiert.
    - Die Reihenfolge der Rückgabewerte entspricht der von `menge1`.
    - Die Funktion verwendet keine Mengenoperationen und ist nicht auf bestimmte
      Datentypen beschränkt.
    """
    if not menge1:
        return []
    elif menge1[0] in menge2:
        return [menge1[0]] + schnittmenge(menge1[1:], menge2)
    else:
        return schnittmenge(menge1[1:], menge2)
```

Die obige Funktion ist mit einem White-Box-Test zu prüfen. Welche Pfade müssen für eine vollständige Prüfung betrachtet werden? Bitte benennen und erläutern Sie diese kurz. **5 Punkte**

Ergänzen Sie in der Datei `prog_ss25/test/test_aufgabe02b.py` die korrespondierenden Tests mithilfe von `pytest`. **3 Punkte**

Aufgabe 3

Gegeben ist die Beispieldatei log.txt, welche eine Log-Datei einer Software darstellt.

Schreiben Sie ein kleines Analyse-Tool für die Kommandozeile (**10 Punkte**)

Anforderungen:

Das Programm akzeptiert folgende **Kommandozeilenargumente**:

- --input oder -i : Name der **Eingabedatei** (Pfad zu einer Textdatei im Log-Format) *(Pflichtargument)*
- --output oder -o : Name der **Ausgabedatei**, in die das Ergebnis geschrieben wird *(Pflichtargument)*
- --befehl oder -b : Gibt den auszuführende Befehl an *(Pflichtargument)*
 - count – Zählt die Anzahl der Zeilen in der Datei
 - errors – Zählt die Anzahl der Zeilen, die das Wort **ERROR** enthalten
 - warnings – Zählt die Anzahl der Zeilen, die das Wort **WARNING** enthalten
- --help oder -h – Gibt eine Hilfsbeschreibung aus und beendet das Programm *(optional)*

Die Wahl der Bibliothek (getopt / argparse) ist Ihnen freigestellt.

Beispielaufruf:

```
python3 aufgabe03.py -i log.txt -o ergebnis.txt -b errors
```

Erwarteter Text in der Datei ergebnis.txt:

```
Befehl: errors  
Ergebnis: 3
```


Aufgabe 4

Wir nutzen eine Baumstruktur, um Wörter anhand ihrer Länge zu sortieren. Dazu erweitern wir den bekannten Binären Suchbaum (BST) zu einem sogenannten Ternären Suchbaum (TST), der folgende Eigenschaften besitzt:

- Wörter mit kürzerer Länge kommen in den linken Teilbaum.
- Wörter mit längerer Länge kommen in den rechten Teilbaum.
- Wörter mit exakt gleicher Länge kommen in den mittleren Teilbaum.

Der Ternäre Suchbaum ermöglicht es, Wörter effizient nach ihrer Länge zu verwalten und bei gleicher Länge gruppiert auszugeben.

Implementieren Sie eine Klasse `TernaererSuchbaum`, die:

- eine Methode `einfuegen(wort)` besitzt, die das Wort entsprechend der oben beschriebenen Regel einfügt,
- eine Methode `inorder_ausgabe()` besitzt, die alle Wörter in Inorder-Reihenfolge der Längen ausgibt.

Tipp: Dabei sollen zuerst alle Wörter aus dem linken Teilbaum, danach aus dem mittleren Teilbaum, dann der aktuelle Knoten und schließlich der rechte Teilbaum ausgegeben werden.

Beispieleingabe:

```
„Cat“      (3 Buchstaben)
„Hello“    (5 Buchstaben)
„Hi“       (2 Buchstaben)
„Dog“      (3 Buchstaben)
„World“    (5 Buchstaben)
„Python“   (6 Buchstaben)
```

Erwarteter Baum:

```
      Cat (3)
     /  |  \
    /   |   \
   Hi(2) Dog(3) Hello(5)
      /  |  \
     /   |   \
    []  World(5) Python(6)
```

Erwartete Ausgabe:

```
Hi, Dog, Cat, World, Hello, Python
```

Gegeben ist der folgende unvollständige Code:

prog_ss25/src/aufgabe04.py

```
class Node:
    def __init__(self, wort):
        self.wort = wort
        self.links = None
        self.mitte = None
        self.rechts = None

class TernaererSuchbaum:
    def __init__(self):
        self.wurzel = None

    def einfuegen(self, wort):
        """
        Fügt ein Wort entsprechend seiner Länge in den ternären Suchbaum ein.
        - kürzer -> links
        - gleich lang -> mitte
        - länger -> rechts
        """
        if self.wurzel is None:
            self.wurzel = Node(wort)
        else:
            self._einfuegen_rekursiv(self.wurzel, wort)

    def _einfuegen_rekursiv(self, aktueller_knoten, wort):
        pass

    def inorder_ausgabe(self):
        """
        Gibt alle Wörter in der Inorder-Reihenfolge der Längen aus:
        links -> mitte -> aktueller Knoten -> rechts
        """
        self._inorder_ausgabe_rekursiv(self.wurzel)

    def _inorder_ausgabe_rekursiv(self, knoten):
        if knoten is not None:
            pass

baum = TernaererSuchbaum()
woerter = ["Cat", "Hello", "Hi", "Dog", "World", "Python"]

for wort in woerter:
    baum.einfuegen(wort)

print("Inorder-Ausgabe der Wörter:")
baum.inorder_ausgabe()
```

- a) Ergänzen Sie die Methode `_einfuegen_rekursiv(self.wurzel, wort)`, die ein neues Element im Baum einfügt **[12 Punkte]**
- b) Ergänzen Sie die Methode `_inorder_ausgabe_rekursiv(self)`, die die Wörter der Länge nach sortiert ausgibt. **[5 Punkte]**

Aufgabe 5

Gegeben ist folgender, leider unvollständiger, Code:

prog_ss25/src/aufgabe05.py

```
def heapsort(input_liste):
    """
    Sortiert die Strings in input_liste aufsteigend
    nach ihrer Länge mithilfe des Heapsort-Verfahrens.
    """
    pass

def heapify(liste, n, i):
    """
    Stellt sicher, dass der Teilbaum mit Wurzel an Index i
    die Max-Heap-Eigenschaft bezüglich der Wortlänge erfüllt.
    """
    groesster = i
    links = 2 * i + 1
    rechts = 2 * i + 2

    if links < n and len(liste[links]) > len(liste[groesster]):
        groesster = links

    if rechts < n and len(liste[rechts]) > len(liste[groesster]):
        groesster = rechts

    if groesster != i:
        liste[i], liste[groesster] = liste[groesster], liste[i]
        heapify(liste, n, groesster)

wort_liste = ["Hallo", "Python", "Wirtschaftsinformatiker", "Aufgabe", "Sortieren"]

print(heapsort(wort_liste))
```

Ergänzen Sie die Funktion `heapsort(input_liste)`, sodass die Liste der Wörter mithilfe des Heapsort-Verfahrens korrekt aufsteigend nach Wortlänge sortiert wird. Nutzen Sie dabei die bereits vorgegebene `heapify`-Funktion. **(12 Punkte)**